

UDUPI DISTRICT ADMINISTRATION · SWACHH BHARAT MISSION

Udupi Civic Watch

Complete Functional & Technical Audit

Purpose: Proposal-ready inventory of the current platform, its user workflows, technical foundation, security posture, limitations, and future opportunities.

Scope: Read-only inspection of the current source code, configuration, tests, API specification, database schema, and documented project state.

Audit date: 11 August 2026

Important limitation: This report describes evidence found in the repository. A code reference, task draft, API declaration, or unused component is not treated as proof that a feature is working in production.

No application code, UI, database, deployment configuration, workflow, secret, or production data was changed during this audit.

Contents

1. Executive Summary
2. Citizen Features
3. Administration Features
4. Department / Officer Workflow
5. User Roles & Permissions
6. Notifications
7. Location / GIS Features
8. Database
9. API & Integrations
10. Technology Stack
11. Security
12. Current Functionality Status
13. Current Limitations
14. Future Enhancement Opportunities
15. Government Project Feature List
16. Project Complexity Assessment
17. Development Effort
18. Final Two-Page Summary

1. Executive Summary

What Udupi Civic Watch is

Udupi Civic Watch is a mobile-first civic cleanliness reporting platform. Residents can report waste issues with a photo and GPS location. The report is assigned into a government-side workflow for allocation, cleanup, monitoring, and closure.

What problem it solves

Waste complaints are often difficult to report, assign, track, and verify. The platform creates a report number, records location and evidence, identifies the relevant service area, assigns responsibility, and lets officials update progress.

Who uses it

- **Citizens:** submit and track reports without an account.
- **Field officers:** view assigned reports, start cleanup, complete work, and upload completion photos.
- **Panchayat administrators and Control Center staff:** monitor reports, officers, status, and local performance.

- **Hierarchy roles:** Commissioner, Environmental Engineer, Health Inspectors, Supervisors, and Community Mobilisers have dedicated accounts or dashboards with varying scope.

Main purpose

To create an auditable, location-aware workflow from a resident's waste complaint to government cleanup action.

What makes it useful

- Combines photo evidence, GPS, ward/service-area checks, routing, status tracking, maps, and official dashboards.
- Supports complaint and cleanup photos, live maps, browser push notifications, and conditional email notifications.
- Provides public citizen services and official workflow tools in one system.

Overall conclusion: The platform has substantial implemented functionality, but it is not ready for an unrestricted public-government launch without security, abuse-prevention, operations, privacy, and recovery improvements.

2. Citizen Features

2.1 Submit a waste report — Implemented

Citizens can create a waste complaint without an account. The report screen requests GPS location, requires at least one photo, uploads evidence, checks the service area, and creates a report.

Collected: latitude/longitude, one or two complaint photos, optional description, optional reporter email, reporter IP for abuse controls, assignment, and timestamps.

Media: photos supported; no evidence of video or document uploads. Location is normally mandatory. New reports trigger official notifications where assignment/configuration permits; a reporter email triggers an attempted acknowledgement.

Tracking: yes. The success screen provides a report number and a direct tracking action.

2.2 Duplicate-report prevention — Implemented

The API checks for an active nearby report from the previous 24 hours and returns the existing report reference when a likely duplicate is found. This is a practical approximate geographic check, not a guarantee of perfect duplicate detection.

2.3 Service-area validation — Implemented

Configured polygon data is used by the client and API to reject locations outside the defined Saligrama or Udupi service zones. Public use is GPS-led; manual placement is primarily test-mode behavior.

2.4 Public report tracking — Implemented, simplified history

Citizens can enter a report number and see Reported, Cleaning, or Cleaned status. The page polls about every 10 seconds and displays complaint/cleanup photos in a lightbox. The visual timeline is derived from current timestamp fields; it is not a complete server-maintained event history.

2.5 My Reports — Implemented locally

The browser stores up to 20 recent report IDs and reloads their public status. This is device-local storage, not a citizen account or cross-device history.

2.6 Public live map — Implemented

The Leaflet/OpenStreetMap map displays status markers, ward/service overlays, report popups, and complaint/cleanup photos. It refreshes periodically and can show a one-time “You Are Here” dot. Continuous walking accuracy and a confirmed one-tap location control are not evidenced. Precise public coordinates, descriptions, addresses, and media require a privacy policy before launch.

2.7 Citizen browser push — Implemented, conditional

Citizens can subscribe to report-linked browser notifications for cleanup started and cleaned events. Delivery depends on browser permission, service-worker operation, VAPID configuration, and network availability.

3. Administration Features

Control Center dashboard

Provides report totals, status summaries, panchayat/ward/officer/date filtering, trends, zone/map views, and performance summaries.

Report management

- Control Center can view, search, filter, inspect, reassign, archive, bulk archive, and manage archived reports.
- Panchayat administrators can view panchayat-scoped reports, filter by status/ward/officer, update status, archive records, and inspect photos.
- Restricted purge/delete capabilities exist for archived records, but retention, approval, reason, and immutable audit controls are not sufficiently evidenced.

Officer management

Control Center can create, edit, deactivate/delete, filter, and assign geographic area information to field officers. Welcome email is attempted if SMTP is configured.

Status management

Officials can move cases through Reported → Cleaning → Cleaned. Cleaning and completion timestamps, cleanup photos, email, in-app, and push notifications are linked to these transitions.

Analytics and reports

Dashboards provide status totals, resolution rates, waste-type patterns, ward/officer/panchayat views, response-time calculations, and conditional AI narrative summaries. Some computations may need database-side optimization as volume grows.

Search and filtering

Search/filtering exists across several official views for status, ward, officer, panchayat, waste type, and text. Reporter-email filtering, complete hierarchy-wide report-number search, and archive-reason filtering are not all confirmed.

Other administration

Role-specific dashboards exist for Commissioner, Environmental Engineer, Health Inspector, Supervisor, and Community Mobiliser. Their confirmed write capabilities vary; several appear primarily read/monitoring-oriented.

4. Department / Officer Workflow

1. **Citizen submits:** photo, GPS, optional description/email.
2. **Validation:** request shape, duplicate check, report rate limit, and service-area check.
3. **Assignment:** coordinates are matched to an officer area; a report may remain unassigned if no match exists.
4. **Notification:** Control Center is notified; assigned officer and relevant panchayat admin are notified where data exists; email is conditional.
5. **Processing:** field officer views work, marks Cleaning, then Cleaned, and can upload cleanup photos.
6. **Citizen update:** public tracking refreshes; email and browser push are conditional.
7. **Closure:** Cleaned is the current resolution endpoint with completion timestamp and optional evidence.
8. **Archival:** archiving/purge is a separate administrative function.

Difference from a full case-management workflow: no strong evidence of formal rejection, escalation, SLA breach, reopen, quality approval, dispute, or immutable action history.

5. User Roles & Permissions

Role	Can see	Create/edit	Delete/archive	Access level
Citizen	Public map and tracking	Create report; subscribe to alerts	Clear local report list	No official access
Field Officer	Assigned reports	Update status; upload completion evidence	No normal delete confirmed	Operational execution
Panchayat Admin	Panchayat-scoped reports, dashboard, analytics	Status and local workflow actions	Archive evidenced	Local oversight
Control Center/Admin	System-wide reports, dashboards, staff	Manage officers; reassign	Archive, bulk archive, restricted purge	Highest general administration
Commissioner	Scoped reports, map, analytics	No normal write workflow confirmed	Test-oriented purge route exists	Senior oversight
Environmental Engineer	Scoped reports, map, analytics	No confirmed CRUD	No confirmed deletion	Monitoring oversight
Health Inspector	Scoped reports, map, analytics	No confirmed CRUD	No confirmed deletion	Monitoring oversight
Supervisor	Dedicated scoped dashboard	No clear CRUD/status workflow confirmed	No confirmed deletion	Limited/read-oriented evidence
Community Mobiliser	Dedicated dashboard	No confirmed report-management workflow	No confirmed deletion	Limited/read-oriented evidence

The API includes role-specific middleware and several geographic-scope tests. The server-side checks are more important than frontend route protection; end-to-end proof is not complete for every role and route.

6. Notifications

Mechanism	Status	Use
In-app	Implemented	Assignments, new reports, status changes, read/clear controls.
Browser push	Implemented, conditional	Citizen report status and official alerts.
Email	Implemented, SMTP-dependent	Acknowledgement, assignment, status, welcome, and weekly digests.
Weekly digest	Implemented in process	Monday 08:00 IST summaries for panchayat admins and Control Center.
SMS	Not evidenced	No provider integration found.
WhatsApp	Not evidenced	No integration found.

Delivery is conditional. There is no evidenced durable delivery queue, retry dashboard, or operator-facing delivery-failure monitor.

7. Location / GIS Features

- GPS capture and coordinate storage.
- Polygon-based service-area validation.
- Udupi/Saligrama ward and geofence data.
- Leaflet/OpenStreetMap interactive maps.
- Public and official report markers.
- Area-based officer assignment and role scope.
- Ward overlays and location-based filtering.
- Navigation links from official report views and email.

This is a practical web mapping/geofence implementation, not an enterprise GIS. No government GIS synchronization, cadastral integration, spatial database optimization, or full GIS editing suite was evidenced.

Location limitations include GPS dependence, one-time rather than continuous user-location display, public precision of report data, and the need for maintained polygon governance.

8. Database

Technology: PostgreSQL with Drizzle ORM and Zod-derived validation.

Entity	Purpose
reports	Complaint, location, photos, status, assignment, timestamps, AI fields, soft deletion.
officers	Field-officer identity, area, panchayat, and lifecycle.
users	Login identities, roles, password hashes, phone/panchayat links.
notifications	Authenticated in-app messages.
push_subscriptions	User/report-linked browser push subscriptions.
password_resets	Password reset records.
conversations/messages	Generic structures; active civic workflow use not confirmed.
Hierarchy tables	Environmental Engineers, Health Inspectors, Supervisors, and Community Mobilisers are created/seeded at runtime.

Stored citizen information: optional email, IP, GPS, description, photos, and push-subscription data when opted in.

Stored complaint information: report ID, images, coordinates, address, description, status, assignment, timestamps, cleanup evidence, waste categories, severity, and analysis timestamps.

Concern: runtime hierarchy-table creation and startup seeding make schema governance, environment consistency, rollback, and recovery harder to control.

9. API & Integrations

Service	Purpose / data flow	Assessment
Express API	Citizen, official, report, auth, notification, upload operations.	Implemented.
OpenAPI/Orval	Typed API contracts and generated frontend clients/schemas.	Implemented.
PostgreSQL/Drizzle	Reports, users, staff, notifications, subscriptions.	Implemented.
Replit Object Storage/GCS client	Complaint and cleanup media persistence.	Implemented, configuration-dependent.
OpenStreetMap/Leaflet	Tiles and geographic visualization.	Implemented.
Nodemailer/SMTP	Citizen and official email notifications.	Implemented, conditional.
Web Push/VAPID	Browser notifications.	Implemented, conditional.
OpenAI-compatible integration	Photo analysis and AI narrative insights.	Partial/conditional.
node-cron	Weekly digest scheduling.	Implemented in process.
Pino	Request/application logging.	Implemented.

10. Technology Stack

Area	Technology
Frontend	React 19, TypeScript, Vite, Tailwind CSS, Radix UI, Lucide, Framer Motion.
Routing/client data	Wouter and TanStack React Query.
Maps	Leaflet, React Leaflet, OpenStreetMap.
Backend	Node.js, Express 5, TypeScript, esbuild.
Database	PostgreSQL and Drizzle ORM.
Validation/API	Zod, drizzle-zod, OpenAPI, Orval.
Authentication	Cookie sessions with bcrypt password hashing.
Media	Replit Object Storage through Google Cloud Storage client.
Messaging	Nodemailer/SMTP and web-push/VAPID.
Testing	Vitest, Supertest, Playwright.
PWA	Vite PWA/Workbox service-worker support.

The repository is configured for Replit artifacts/workflows, but a complete production deployment, backup, rollback, and disaster-recovery definition was not found.

11. Security

Implemented controls

- Bcrypt password hashes.
- Role-specific API authorization middleware.
- Geographic scope checks in multiple routes and tests.
- Zod request validation.
- Report rate limit and duplicate checks.
- Service-area rejection.
- Filename sanitization on media retrieval.
- Request logging.
- Soft deletion in selected flows.

Launch-critical weaknesses

Critical — default credentials and fallback secret: startup behavior can create/reset a known control-center credential, and authentication has a fallback secret if configuration is missing.

Critical — custom session signing: the session construction is custom rather than a standard reviewed authenticated-session mechanism.

Critical — public uploads: the upload route is public and lacks evidenced content inspection, robust allow-listing, pixel/dimension limits, upload quotas, scanning, orphan cleanup, and explicit privacy controls.

High — permissive credentialed CORS: production origins need strict allow-listing and CSRF review.

High — public privacy exposure: precise locations, descriptions, addresses, and media may be exposed through the public map.

High — limited abuse controls: login, tracking, upload, password reset, and other public endpoints do not show comprehensive centralized throttling.

High — operational gaps: health checks do not verify database/storage/push/scheduler readiness; no tested backup/restore, centralized error monitoring, or alerting was evidenced.

Security conclusion: The authorization foundation is promising, but the system should be treated as pre-production until the critical and high-risk findings are addressed and independently tested.

12. Current Functionality Status

A. Fully implemented and evidenced

- Citizen reporting with photo, GPS, optional description/email.
- Service-area validation, duplicate check, and basic rate control.
- Public tracking and three-state lifecycle.
- Field cleanup photo workflow.
- Multi-photo public/official map and tracking galleries.
- Public live map and ward/service overlays.
- Local recent-report list.
- Control Center, panchayat, officer, and hierarchy dashboards.
- Role-aware API checks in several routes.
- In-app notifications, browser push infrastructure, SMTP email, weekly digest.
- PostgreSQL and object-storage persistence.
- Focused API and Playwright tests.

B. Partially implemented / needs attention

- Status timeline is simplified rather than a full event history.
- My Reports is device-local rather than account-based.
- AI is conditional and not a complete operational workflow.

- Hierarchy roles have uneven confirmed write capabilities.
- Manual location fallback is test-mode-oriented.
- Notification delivery lacks durable retry/monitoring evidence.
- Analytics may need database optimization.
- Health checks, startup schema behavior, archival governance, and PWA/offline behavior need strengthening.

C. Referenced but not confirmed end-to-end

- SMS and WhatsApp.
- Enterprise government GIS integration.
- Complete AI backfill, citizen AI findings, and AI severity workflow.
- Formal archive reasons and citizen-facing archive explanations.
- Continuous user-location tracking and one-tap map relocation.
- Complete immutable report event history.
- SLAs, escalation, quality approval, reopening, and dispute workflows.
- Backup/restore, disaster recovery, monitoring, alerting, and deployment rollback.
- Video/document uploads and cross-device citizen accounts.

13. Current Limitations

- **Scalability:** some analytics and map/list operations may become expensive as volume increases.
- **Security:** default credential/session, upload, CORS, privacy, and rate-control risks require action.
- **Reliability:** external notifications are often fire-and-forget; dependency health is lightly checked.
- **Data protection:** no documented tested backup/restore, media lifecycle, or privacy/retention program was found.
- **Citizen experience:** GPS dependence, basic report search, device-local history, and limited accessibility/local-language evidence.
- **Administration:** limited escalation, approval, quality-review, reopening, and audit-history capabilities.
- **Infrastructure:** no complete repository-defined production monitoring, rollback, or incident response path.

14. Future Enhancement Opportunities

Immediate improvements before launch

1. Remove default credentials and enforce secure production secrets.
2. Harden sessions, cookies, CORS, CSRF, and password-reset behavior.
3. Secure uploads with validation, scanning, limits, quotas, cleanup, and privacy rules.

4. Add dependency health checks, centralized error monitoring, metrics, alerts, and incident procedures.
5. Establish backup, restore, retention, and migration rollback processes.
6. Define public-map redaction and consent policy.
7. Add complete end-to-end tests for citizen, role, status, notification, and failure paths.
8. Complete accessibility, local-language, and legal-notice review.

Phase 2

- Citizen accounts and cross-device history.
- Formal report event history.
- SLAs, escalation, overdue queues, supervisor approval.
- Reopen/dispute/quality review.
- Archive reasons, retention, and records management.
- SMS/WhatsApp where appropriate.
- Multilingual interfaces and better search.
- Offline field work with conflict-safe synchronization.
- Database-driven analytics optimization.

Advanced/future

- Government GIS interoperability.
- Sensitive-location privacy zones and media redaction.
- Workload/proximity-aware routing.
- Human-reviewed computer vision.
- Interoperability with sanitation, engineering, grievance, and call-center systems.
- Privacy-preserving public transparency dashboards.
- IVR, SMS, WhatsApp, and assisted reporting.
- Waste-hotspot forecasting and resource planning.
- Digital compliance audit reporting.

15. Government Project Feature List

Citizen Services

- Mobile waste reporting without account creation.
- Camera photo evidence and GPS location.
- Optional email updates and browser alerts.
- Report-number status tracking.
- Recent-report list on the same device.

Complaint & Issue Reporting

- Unique report reference number.
- Location, description, and visual evidence.
- Duplicate prevention and service-area validation.

- Before-and-after cleanup evidence.
- Reported, Cleaning, and Cleaned status.

Department Administration

- Automatic routing to responsible staff by area.
- Field officer work queues.
- Assignment and reassignment.
- Officer roster and area management.
- Panchayat and Control Center oversight.
- Role-based official access.

Monitoring & Tracking

- Live public cleanliness map.
- Ward/service-area visualization.
- Cleanup progress monitoring.
- Map-based report inspection.
- Official dashboards.

Notifications

- Officer assignment alerts.
- Official status alerts.
- Citizen email acknowledgement where provided.
- Citizen browser cleanup alerts.
- Weekly official summaries.

Reports & Analytics

- Status totals and resolution rates.
- Ward, officer, and panchayat views.
- Response-time indicators.
- Waste-pattern trends.
- Geographic visualization.
- Optional AI-assisted summaries.

User Management

- Official login.
- Field officer management.
- Panchayat and district hierarchy accounts.
- Password-change and activation flows.

Infrastructure & Security

- Centralized report database.
- Bcrypt password hashing.
- Role and geographic access controls.

- Digital image storage.
- Request logging.
- Soft archival in selected workflows.

16. Project Complexity Assessment

Assessment: Advanced application, progressing toward a production-grade civic technology platform.

This is substantially more than a basic form or CRUD dashboard. It combines public reporting, GPS/geofencing, maps, multi-photo media, role-specific government workflows, assignment, status lifecycle, dashboards, analytics, PostgreSQL, object storage, email, push, scheduling, API generation, AI capability, and automated tests.

It should not yet be described as production-grade public civic infrastructure because security hardening, privacy, backup/recovery, monitoring, formal deployment governance, and end-to-end assurance remain incomplete.

17. Development Effort

Estimated build-from-scratch effort: approximately 1,800–3,200 developer hours.

Work area	Indicative effort
Requirements, workflow design, UX	180–320 hours
Citizen reporting, tracking, maps, PWA	300–520 hours
Backend APIs, validation, routing, media	320–540 hours
Official dashboards and role workflows	360–650 hours
Database and API contracts	160–280 hours
Email, push, scheduled notifications	140–260 hours
Analytics and AI-assisted summaries	160–320 hours
Testing, QA, accessibility, bug fixing	250–450 hours
Deployment, security, operations	280–520 hours

Assumptions: experienced small team; responsive web/PWA rather than separate native apps; managed PostgreSQL/storage/email/push; limited initial geography; no major ERP/GIS/call-center integration; public-launch security, privacy, accessibility, monitoring, backup, and onboarding included.

Interpretation: the lower end represents a controlled pilot. The higher end is more realistic for public-government readiness.

Available evidence about time spent in Replit

No verified hour total was available. The workspace does not expose a reliable Replit time-tracking counter to this audit. The available project/task history shows observable activity from **24 April 2026 through 11 August 2026**, a span of **109 calendar days**.

This calendar span is not equivalent to hours worked. It includes no reliable information about session duration, breaks, concurrent work, work outside Replit, or time spent by other contributors. It must not be presented as the user's actual hours spent building the application.

PDF-safe statement: “Verified hours spent in Replit: not available from the inspected workspace. Observable project activity window: 109 calendar days (24 April–11 August 2026).”

18. Final Two-Page Summary

Udupi Civic Watch is a digital public-cleanliness reporting system designed to connect citizens with local government cleanup teams. Residents can report waste issues from a mobile device by taking a photo and allowing the application to capture their GPS location. The system gives the complaint a report number, checks whether the location falls within the service area, tries to identify the responsible field officer, and allows the citizen to follow progress.

The citizen-facing service is straightforward: report waste, attach photo evidence, receive a report number, and track the complaint as it moves from Reported to Cleaning to Cleaned. Citizens may optionally provide an email address for acknowledgements and status updates, and may subscribe to browser alerts for cleanup progress. A public map shows reported issues and their progress, while available complaint and cleanup photos can be viewed.

For government users, the platform provides operational dashboards and role-based access. Field officers can see assigned work, begin cleanup, mark work complete, and upload evidence. Panchayat administrators can monitor local reports, review officers and status, and archive records. The Control Center can view system-wide reports, manage field officers, filter and search data, reassign work, monitor analytics, and manage archived records. Dedicated dashboards also exist for Commissioner, Environmental Engineer, Health Inspector, Supervisor, and Community Mobiliser roles.

The platform includes geographic service boundaries, ward overlays, officer-area assignment, report maps, analytics by status and area, email notifications, browser push notifications, and scheduled weekly summaries. It also has a foundation for AI-assisted operational summaries and waste-photo analysis, though those features should be treated as optional and still maturing.

Technically, the platform is a modern web application built with React, Node.js, PostgreSQL, maps, cloud image storage, email, browser push messaging, and automated testing. It is more capable than a simple complaint portal because it combines citizen reporting, location-aware routing, media evidence, official workflows, notification tools, and analytics in one system.

The current application is best described as an **advanced civic reporting platform suitable for pilot or controlled operational use after security improvements**. It is not yet fully ready for an open public-government launch. The most urgent requirements are to remove unsafe

default account/session behavior, harden official login and browser sessions, secure public uploads, control media/privacy exposure on the public map, introduce stronger rate limiting, add full monitoring and health checks, and establish tested backup/recovery procedures.

With these improvements, Udupi Civic Watch can become a strong operational platform for improving cleanliness reporting, accountability, transparency, and response coordination across Udupi's wards and local bodies.

Audit basis: source code, project configuration, API specification, database schema, tests, task metadata, and the supplied audit brief inspected on 11 August 2026.

Confidentiality: no passwords, API keys, tokens, secrets, credentials, or personally sensitive production records are included.